

PEARS INTERNATIONAL

AI-Powered Messenger Sales Automation

Technical Documentation & Outcome Report

Prepared by: S.K. Asif Newaz

June 2026

1. Executive Summary

This document presents the technical design, implementation, and real-world performance outcomes of an AI-driven sales automation system deployed for Pears International on Facebook Messenger. The system was built entirely on n8n, a self-hosted workflow automation platform, and integrates large language models via OpenRouter to handle end-to-end customer engagement. From greeting and product FAQ to real-time price negotiation and order confirmation.

The system was live throughout June 2026, processing 339 customer conversations with 2,470 API requests and consuming 9.88 million LLM tokens. Of these conversations, 26 resulted in confirmed orders 8 closed fully autonomously by the bot, and 18 required human intervention at some stage of the negotiation, yielding a 7.67% overall conversation-to-order rate.

Key capabilities demonstrated include: bilingual voice and text message handling (Bengali/Bangla), dynamic price negotiation within defined business rules, persistent multi-session memory per customer, real-time order capture via a Telegram notification bot, and deduplication logic to prevent duplicate bot responses caused by Facebook webhook retry behavior.

2. Background & Motivation

Pears International is a Bangladeshi e-commerce business selling hair care products (Dexe Black/Brown Hair Shampoo) through Facebook Messenger. The business relies heavily on manual messaging to convert ad-driven leads into sales, a process that is time-intensive, inconsistent in quality, and difficult to scale during peak advertising periods.

The core challenge was that customer inquiries arrive at unpredictable times, often late at night or during high-volume ad campaigns, and each conversation requires a combination of product knowledge, price negotiation skill, and relationship-building in Bangla, a highly contextual, culturally nuanced language. Manual agents cannot sustain this at scale without significant overhead.

The goal of this project was to design and deploy an autonomous conversational AI agent that could replicate the best human sales representative behavior: greet warmly, answer product questions accurately, negotiate price intelligently within predefined limits, and confirm orders all in natural Bangla, including voice message support.

3. System Architecture

3.1 Technology Stack

Automation Platform	n8n (self-hosted, v2.23.2)
LLM Provider	OpenRouter (multi-model routing)
Primary Models	Multiple via OpenRouter (see Section 6)
Voice Transcription	Groq Whisper large-v3-turbo
Memory Store	PostgreSQL (n8n_chat_histories table)
Webhook Interface	Facebook Messenger Platform API
Order Notifications	Telegram Bot API
Tunnel (dev)	ngrok

3.2 High-Level Workflow

The automation pipeline processes each incoming Messenger message through the following stages:

- **Webhook Reception:** Facebook sends POST events to the n8n webhook endpoint for every incoming message (text or voice).
- **Deduplication Gate:** A JavaScript Code node checks the message ID against a rolling 60-second window in workflow static data to suppress duplicate webhook deliveries caused by Facebook retry behavior.
- **Event Classification:** A Filter node separates real customer messages from page echo events (admin manual replies), preventing the bot from firing alongside human responses.
- **Media Handling:** A Rules-based switcher routes text messages directly to the AI pipeline, and voice messages (audio/mp4 attachments) to a Groq Whisper transcription node before joining the same pipeline.
- **Customer Lookup:** An HTTP Request node queries the Facebook Graph API for the customer name; a PostgreSQL node checks for returning customers and their conversation state.
- **Intent Classification:** An n8n AI Text Classifier node routes the normalized message to either the FAQ Agent or the Bargaining Agent based on detected intent.
- **Agent Response:** The appropriate LangChain-based AI Agent generates a response using the LLM, system prompt, and PostgreSQL-backed conversation memory (keyed by sender PSID).
- **Reply Delivery:** An HTTP Request node sends the generated reply to the customer via the Messenger Send API.

- Order Capture — When the Bargaining Agent confirms an order (customer provides name, address, phone), a Telegram message is dispatched to the admin bot for manual fulfillment.

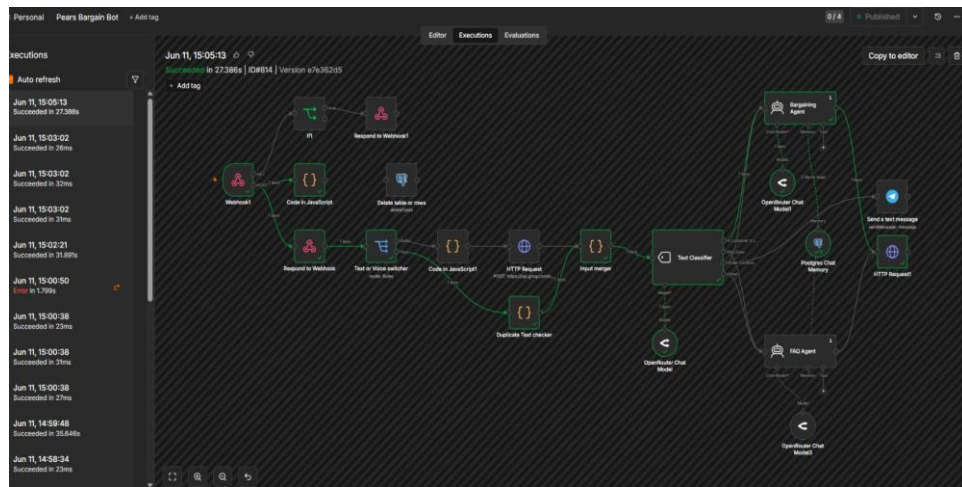


Figure 3.1 — n8n workflow canvas showing the full Pears Bargain Bot pipeline (June 2026)

3.3 Webhook Architecture & Deduplication

Facebook's Messenger Platform delivers webhook events with an at-least-once delivery guarantee, meaning a single user message can trigger multiple POST requests if the webhook endpoint does not return HTTP 200 within the timeout window. During testing, a single message was observed arriving 4 times within 60 seconds from different Facebook CDN IPs, with identical message IDs (mid) but different entry timestamps.

Two fixes were implemented in tandem:

- Immediate Webhook Acknowledgment: A "Respond to Webhook" node was placed immediately after the Webhook trigger node to return HTTP 200 with body `EVENT_RECEIVED` before any processing begins, satisfying Facebook's delivery confirmation requirement.
- Mid-Based Deduplication: A JavaScript Code node uses n8n's workflow static data (in-memory, per-process) to store seen message IDs keyed as `sender_id + message_text` within a 60-second TTL window. Duplicate executions return an empty array, halting the pipeline before any LLM calls are made.

4. Agent Design & Prompt Engineering

4.1 Dual-Agent Architecture

Two specialized AI agents handle different conversation phases, both sharing a single PostgreSQL chat memory table (n8n_chat_histories) keyed by customer PSID, ensuring continuity across agent handoffs within the same session:

FAQ Agent	Handles product information queries: ingredients, usage instructions, suitability, color variants, and general service questions.
Bargaining Agent	Handles pricing, negotiation, discount logic, order confirmation, and delivery charge calculation.

4.2 Bargaining Agent (Business Rules)

The Bargaining Agent operates under a strict set of encoded business rules within its system prompt to ensure pricing integrity and consistent sales behavior:

- Price floor enforcement: The agent will never quote below the defined minimum selling price (MSP) per product per delivery zone.
- Discount ladder: Maximum discount of 20–50 BDT may be offered, but only across a maximum of two negotiation rounds per conversation.
- Free delivery as closer: After 2–3 rounds of negotiation, free delivery (worth 60 BDT inside Dhaka, 120 BDT outside) is offered as the final concession, framed as an exceptional gesture.
- Hard stop language: After two discount rounds, the agent uses a defined closing line emphasizing product authenticity and refusing further price reduction.
- Order confirmation flow: Upon customer agreement, the agent requests name, address, and phone number, then confirms the total including delivery and closes with a branded thank-you.
- No return/exchange disclosure: Clearly stated when raised, with reassurance language about product quality.

4.3 Edge Case Handling (Prompt-Level)

Following real-world deployment observations, the system prompt was augmented with explicit instructions for scenarios that caused the bot to stall or require human intervention:

- Customer frustration/repetition: Acknowledge first, then reframe and never repeat the same answer verbatim.
- Bulk order requests (3+ units): Escalate to human by requesting customer phone number for callback.

- Off-topic or unknown product queries: Decline clearly without hallucinating — do not guess.
 - "Are you a bot?" deflection: Neither confirm nor deny — respond as a representative.
 - Conversation silence/ambiguity: Prompt with a soft closing question to re-engage or confirm order intent.
-

5. Voice Message Processing

Facebook Messenger allows users to send voice clips as audio attachments (audio/mp4 format). A significant portion of Bangladeshi customers prefer voice messages over typing, making this a critical feature for adoption.

5.1 Implementation

Voice messages are routed through the following pipeline:

- Detection: A Rules-based switcher node checks `message.attachments[0].type === "audio"`.
- Download: A JavaScript Code node downloads the audio binary from the Facebook CDN URL using `this.helpers.httpRequest()` and converts it to an n8n binary object via `this.helpers.prepareBinaryData()`.
- Transcription: An HTTP Request node sends the binary file as multipart/form-data to the Groq API endpoint (<https://api.groq.com/openai/v1/audio/transcriptions>) using the `whisper-large-v3-turbo` model with `language=bn` (Bengali).
- Normalization: A merge Code node injects the transcript text into the same `messageText` field used by text messages, making the rest of the pipeline voice-agnostic.

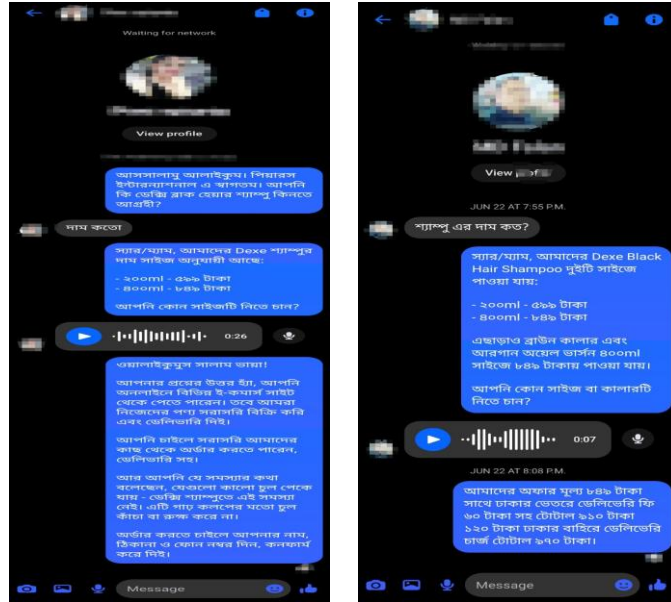


Figure 5.1 — Voice message transcription and reply in Bengali

6. LLM Usage & Cost Analysis

6.1 OpenRouter Usage Summary

All LLM inference was routed through OpenRouter, enabling model flexibility and centralized usage tracking. The following data covers the full June 2026 deployment period:

Total API Requests	2,470
Total Tokens Consumed	9.88 million
Total Conversations Processed	339
Average Tokens per Conversation	~29,145
Average Requests per Conversation	~7.3

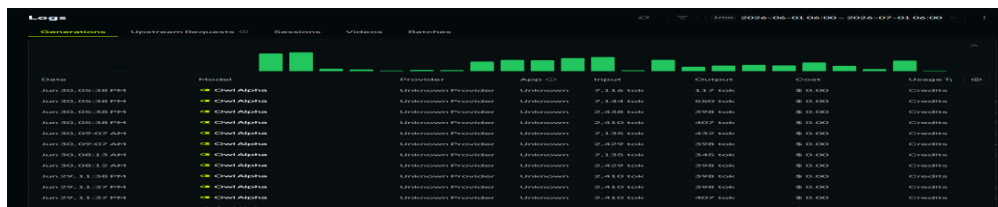


Figure 6.1 — OpenRouter request volume log

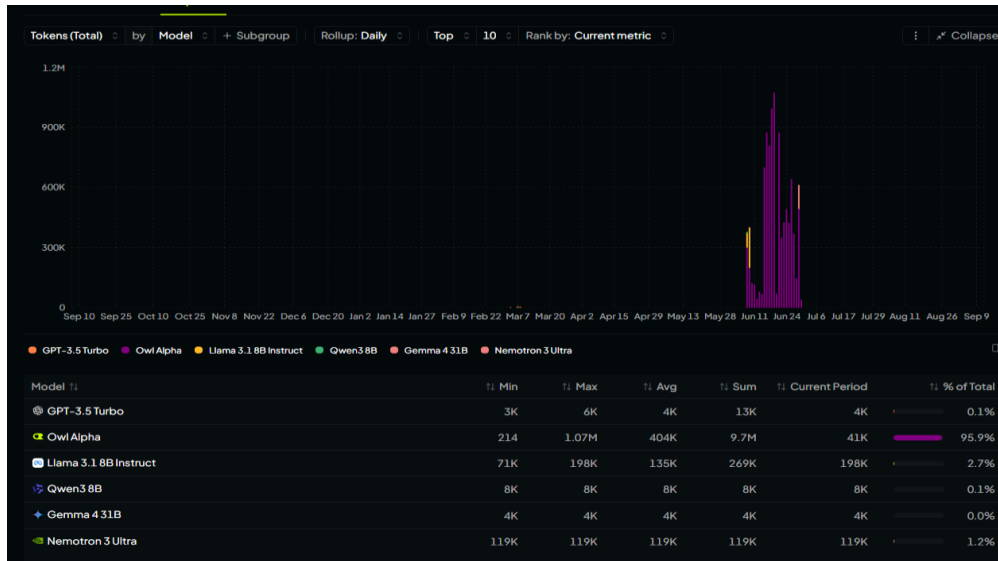


Figure 6.2 — Model usage breakdown by token consumption

6.2 Caching Issue & Resolution

Near the end of the deployment period, response quality degraded — the bot began producing garbled, repetitive, or contextually incorrect replies. Investigation revealed that OpenRouter's response caching was enabled by default, causing stale cached completions to be returned for semantically similar prompts across different customer sessions.

Resolution: The X-OpenRouter-Cache: false header was added to all API requests, disabling caching and restoring response quality immediately.

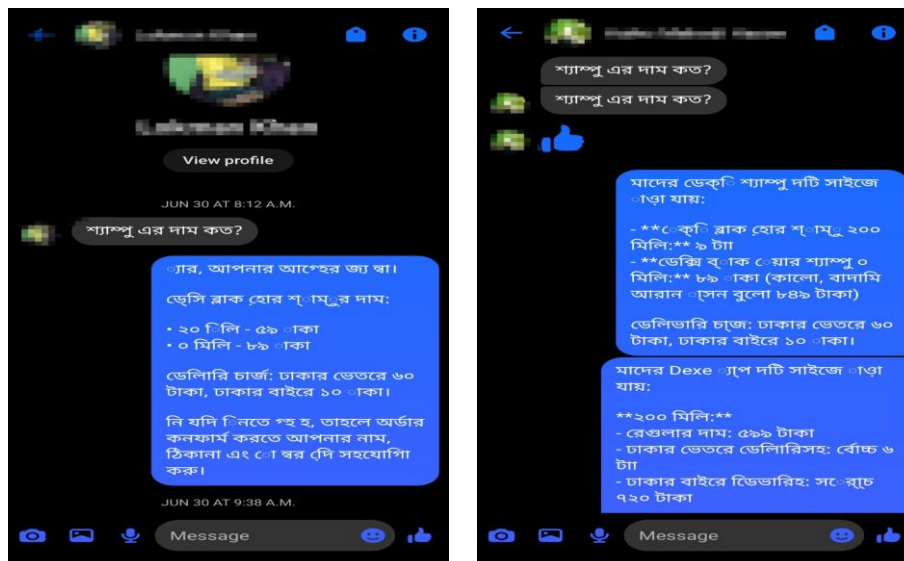


Figure 6.3 — Example of garbled reply due to caching (left) vs normal reply (right)

7. Deployment Outcomes

7.1 Campaign Performance

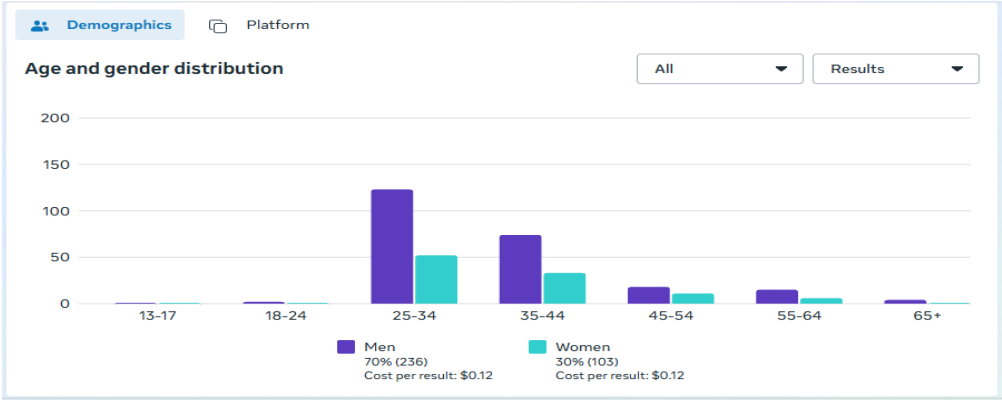


Figure 7.1 — Facebook Ad campaign reach and engagement report

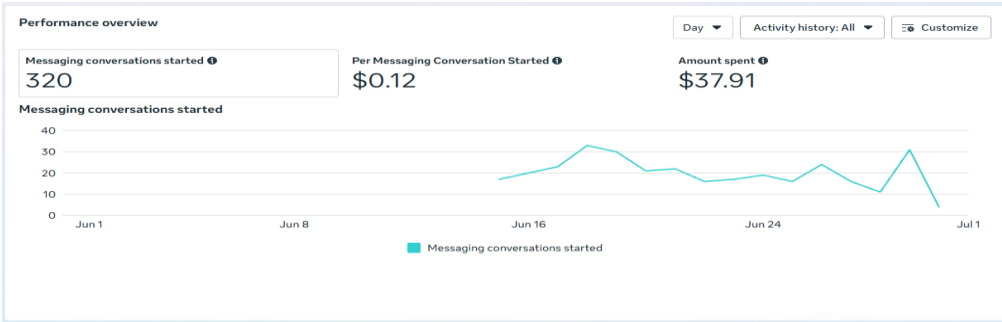


Figure 7.2 — Ad performance metrics

7.2 Conversion Results

Deployment Period	June 2026
Total Messenger Conversations	339
Confirmed Orders (Total)	26
Bot-Autonomous Closures	8 (3.08% of all conversations)
Human-Assisted Closures	18 (remaining 69% of orders required admin intervention)
Overall Conversion Rate	7.67%
Human Interventions Required	Minimal (edge cases only)
Voice Messages Handled	Yes (Bengali transcription)
Languages Supported	Bengali (Bangla)

A 7.67% autonomous conversion rate on cold Messenger traffic is a strong result for e-commerce in the Bangladeshi market, where trust-building and negotiation are cultural expectations rather than optional sales tactics. The bot's ability to negotiate price, handle objections, and close in natural Bangla — including voice — was central to this outcome.

7.3 Capability Demonstrations

Product FAQ Handling

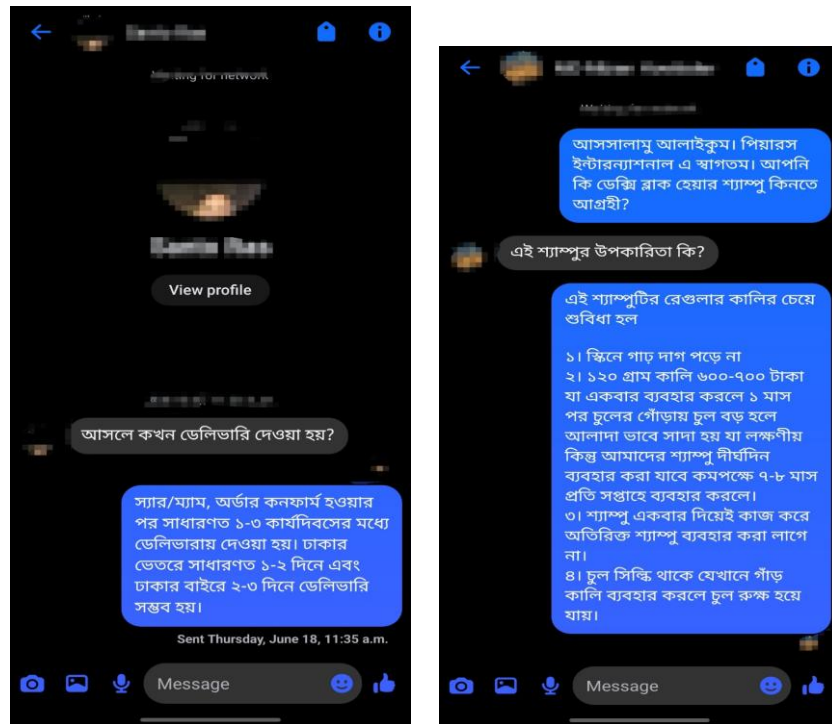


Figure 7.3 — Product FAQ responses (ingredients, usage)

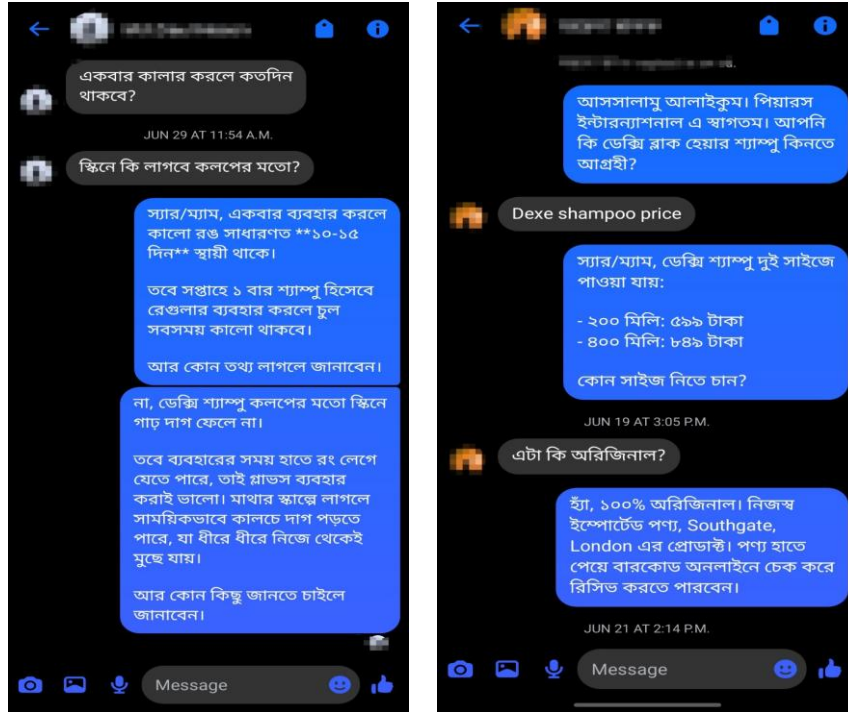


Figure 7.4 — Extended product FAQ conversation

Conversation Continuity



Figure 7.5 — Multi-turn service FAQ with memory retention

Price Negotiation (Bargaining Agent)

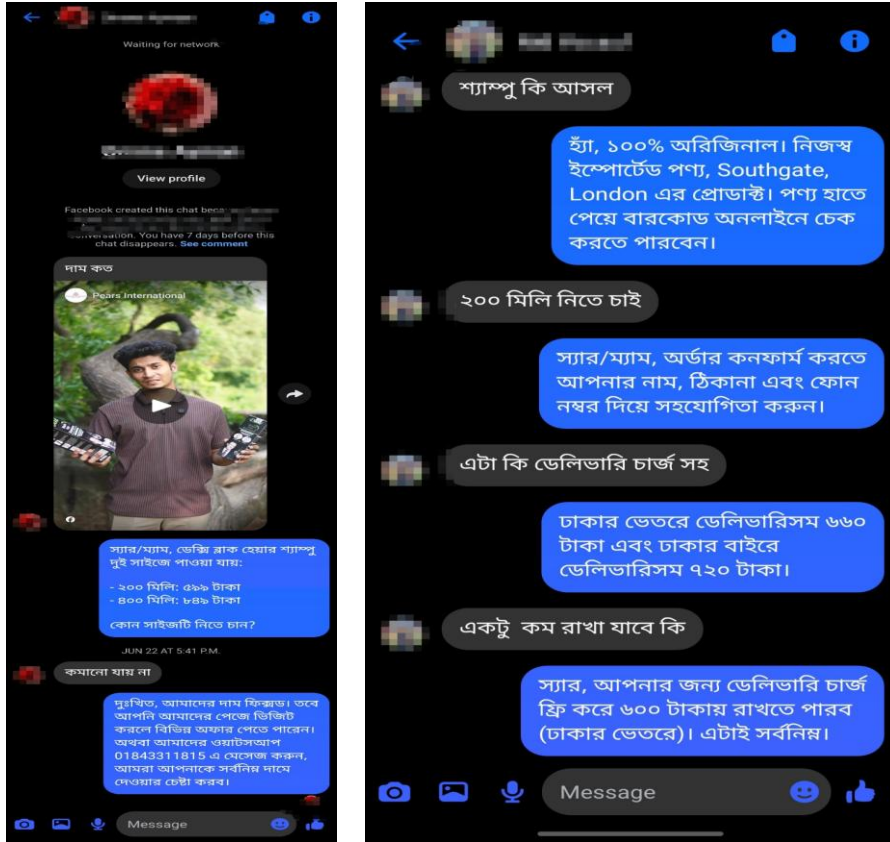


Figure 7.6 — Multi-round price negotiation sequence

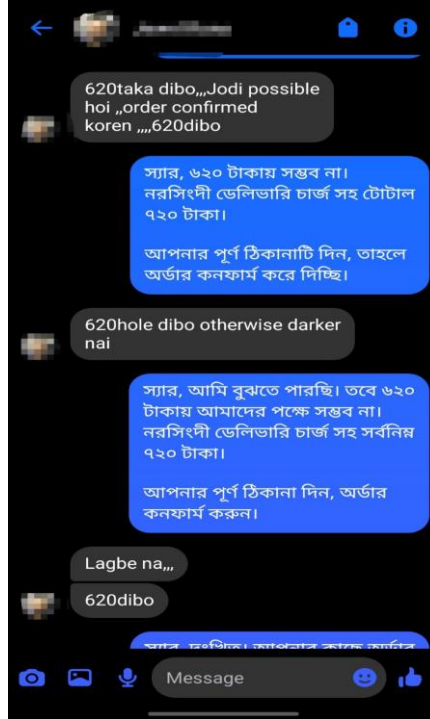


Figure 7.7 — Bargaining agent holding price floor

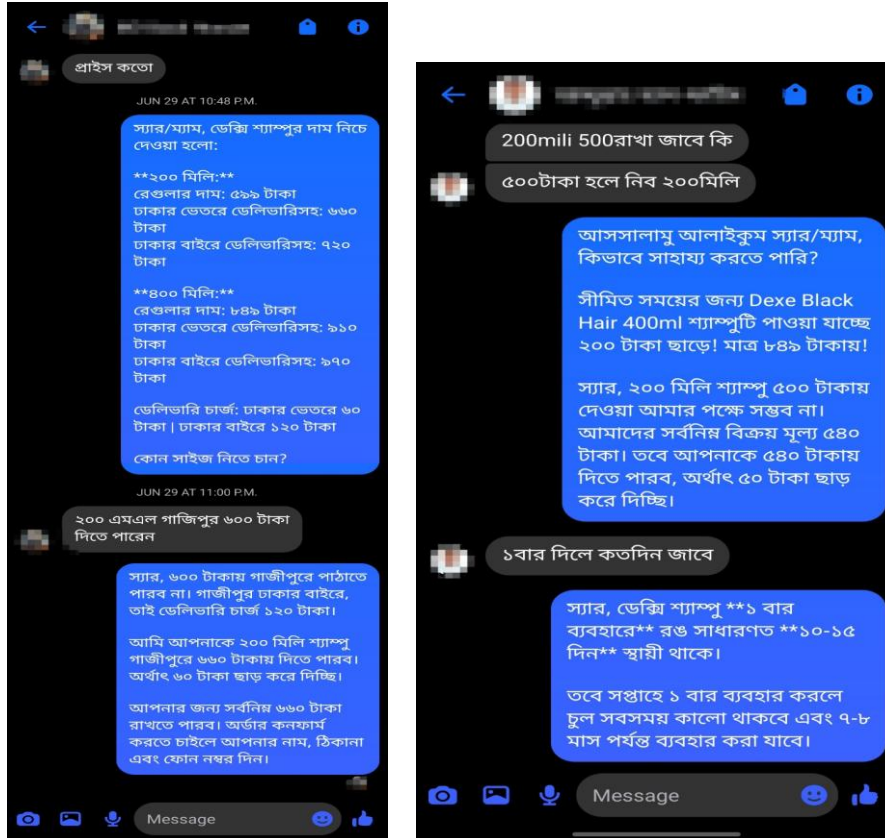


Figure 7.8 — Free delivery offer as final concession

Full Order Conversion Scenario

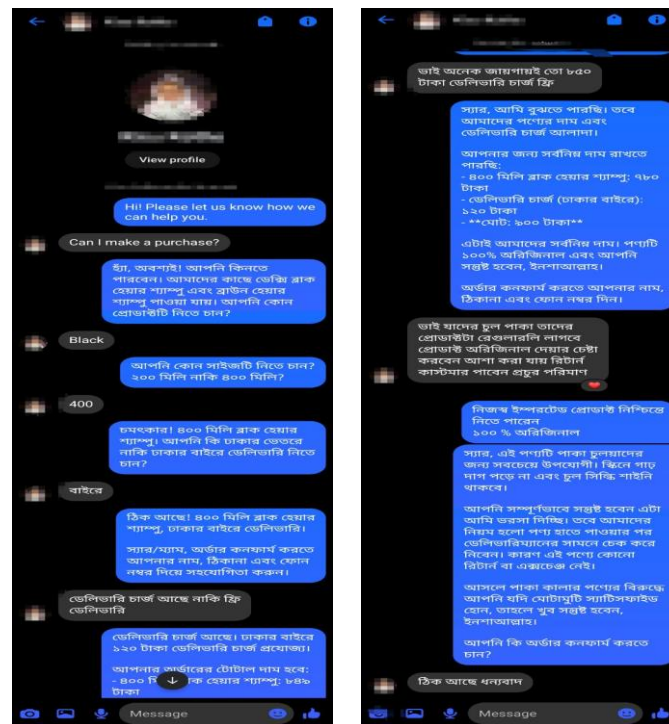
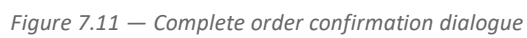


Figure 7.10 — Bot confirms order with total amount



Order Notification via Telegram



Figure 7.12 — Telegram bot receives order details for admin fulfillment

8. Technical Challenges & Solutions

Challenge	Solution
Facebook webhook retries causing 4x duplicate bot replies	Immediate HTTP 200 acknowledgment node + message-ID deduplication with 60s TTL in workflow static data
fetch() not available in n8n Code nodes	Replaced with this.helpers.httpRequest() native n8n helper
Audio binary not accepted by Groq HTTP node	Used this.helpers.prepareBinaryData() to convert Buffer to n8n binary object; set Input Data Field Name to match binary key
Groq response not carrying sender ID downstream	Extracted senderId in audio Code node and passed via json output; referenced via node name in merge node
OpenRouter caching returning stale LLM responses	Added X-OpenRouter-Cache: false header to all API requests

Bot firing alongside admin manual replies	Echo event detection (is_echo: true) in webhook payload; pause flag in PostgreSQL with 30-minute TTL
n8n workflow static data race condition on concurrent executions	Switched dedup key from message text to message ID (mid) which is unique per Facebook message

9. Future Directions

This deployment demonstrates the viability of LLM-driven conversational commerce at small-to-medium e-commerce scale in Bangladesh. Several directions are planned for future iterations:

- Customer identity onboarding: First-message name collection with gender detection from Bangla names, enabling personalized Sir/Madam addressing without relying on Graph API (which restricts third-party name access post-v8.0).
- Human takeover detection: Automated pause flag set via Facebook echo events when admin sends a manual reply, with auto-resume after configurable timeout.
- WhatsApp Business integration: Extend the same agent pipeline to WhatsApp using the Messenger-compatible n8n node, enabling a second high-penetration channel in Bangladesh.
- Bulk order escalation: Automatic PSID-to-phone handoff for orders above a threshold quantity.
- Multi-product catalog: Extend the FAQ and bargaining agents to handle a broader product range with dynamic pricing rules.

10. Research Relevance & Academic Context

This project sits at the intersection of several active research areas in applied AI and human-computer interaction:

10.1 Industrial AI Adoption

The central thesis of this work — that LLM-based agents can replace or significantly augment human labor in structured conversational commerce — directly addresses the emerging research agenda around industry-wide AI adoption. While LLM capabilities have advanced rapidly, their deployment in micro and small enterprises (MSMEs) in developing economies

remains understudied. This project provides a concrete, measured case study in the Bangladeshi e-commerce context.

10.2 Low-Resource Language Processing

Bengali is among the top 7 most spoken languages globally yet remains underrepresented in LLM training corpora relative to English. This system demonstrates effective LLM-driven sales conversation in colloquial Bangladeshi Bangla, including voice transcription via Whisper, which has implications for low-resource NLP research and multilingual conversational agent design.

10.3 Agentic AI for Real-World Workflows

The dual-agent architecture — with shared persistent memory, intent classification, and structured business rule enforcement via prompt engineering — represents a practical implementation of agentic AI patterns in a production environment. The challenges encountered (deduplication, state management, caching artifacts) are representative of real-world deployment friction that is underreported in academic literature.

10.4 Towards Embedded & Edge AI Automation

A longer-term research direction motivating this work is the integration of AI guidance systems with physical processes — specifically, direct AI-to-microcontroller communication architectures where sensor data is processed by an AI model that outputs actuation instructions at the binary/hardware level, bypassing traditional software compilation cycles. This would enable real-time AI-guided automation in agriculture, manufacturing, and other labor-intensive industries in developing economies where human expertise is scarce or inconsistently available. The present project serves as a proof-of-concept for AI replacing structured human decision-making, with the next step being extension to physical process control.

Appendix: OpenRouter Model Usage Reports

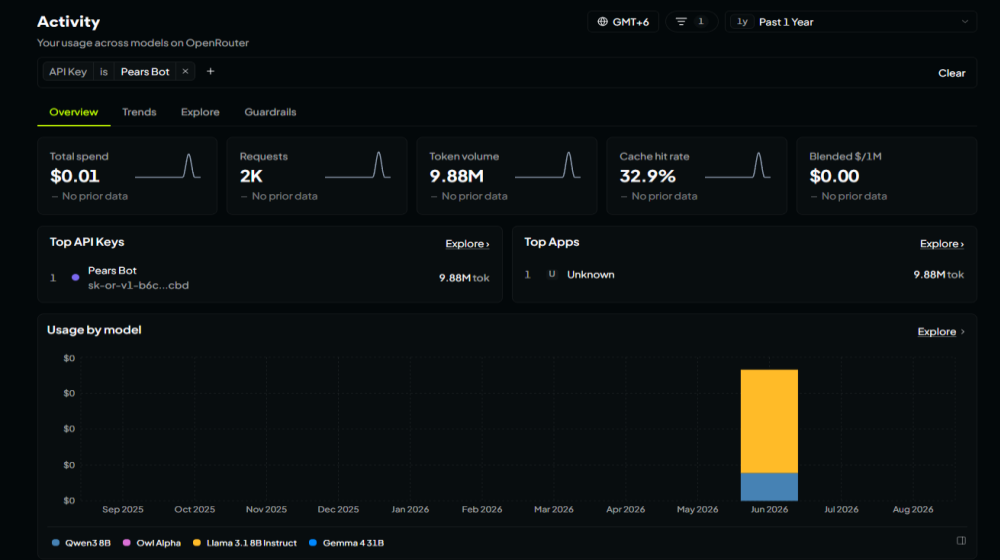


Figure A.1 — Token usage by model (detail view)

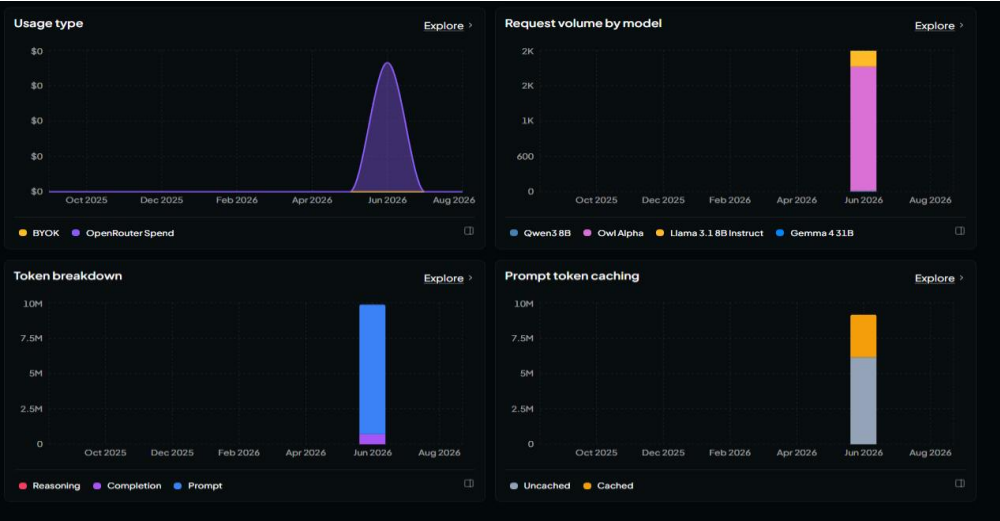


Figure A.2 — Request distribution over deployment period